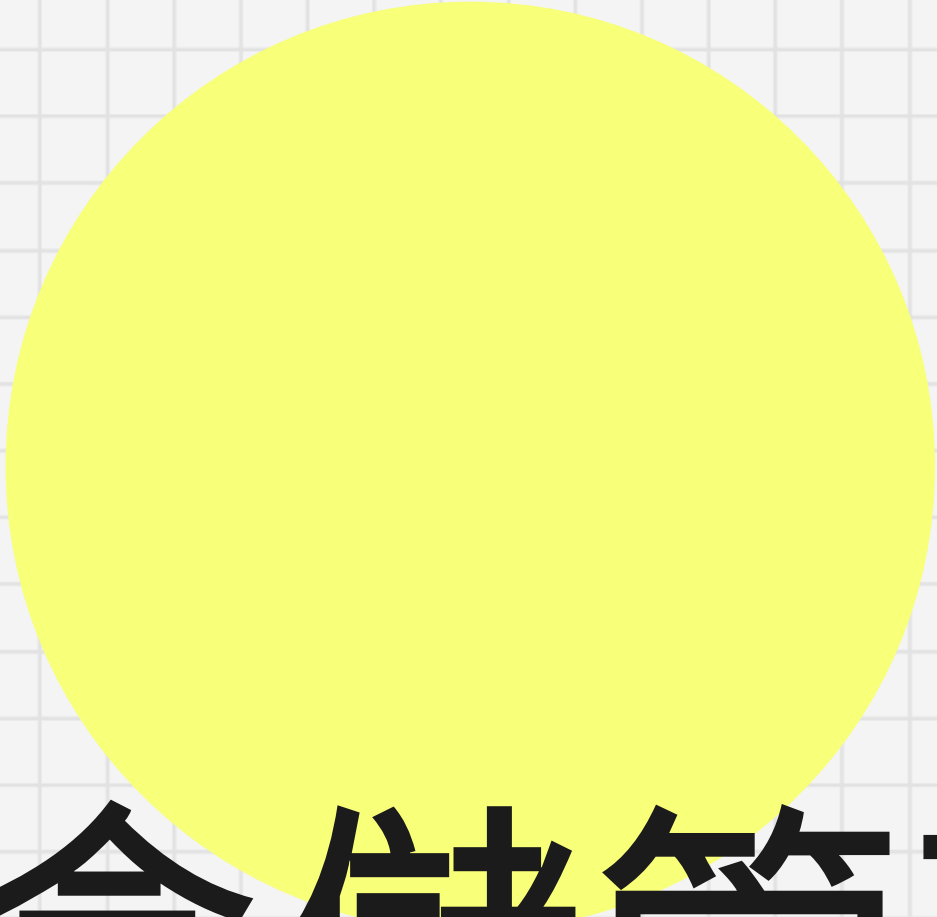
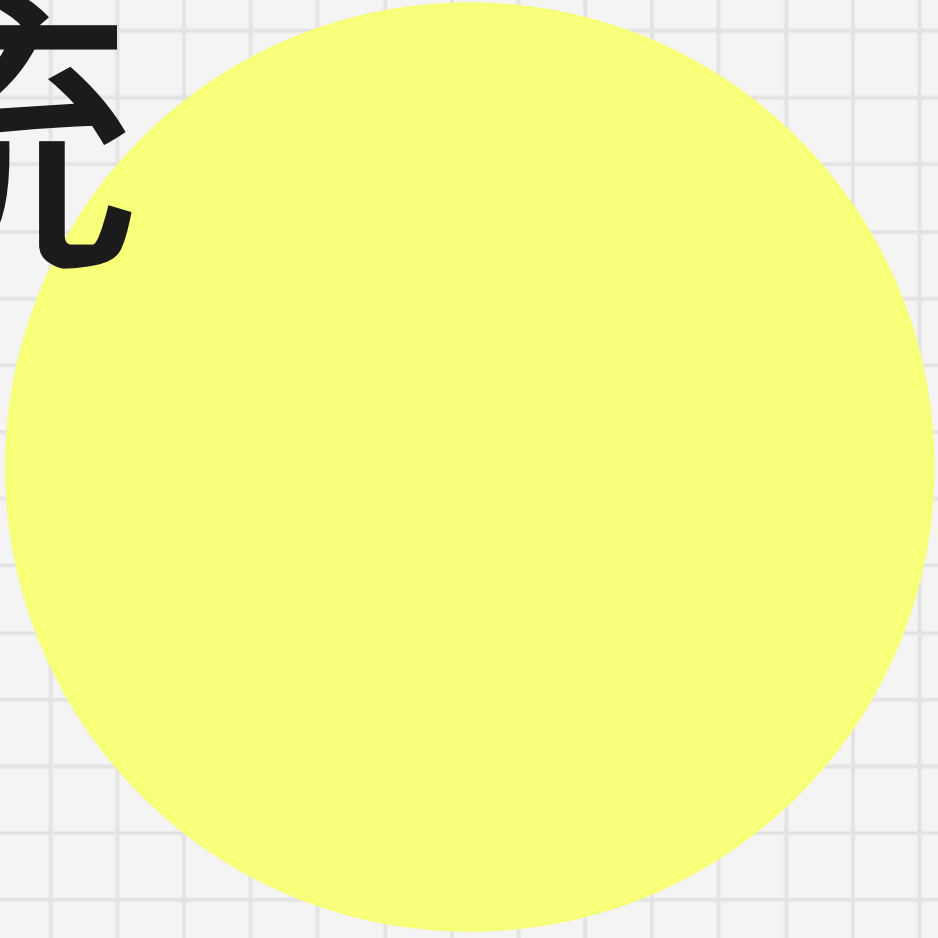
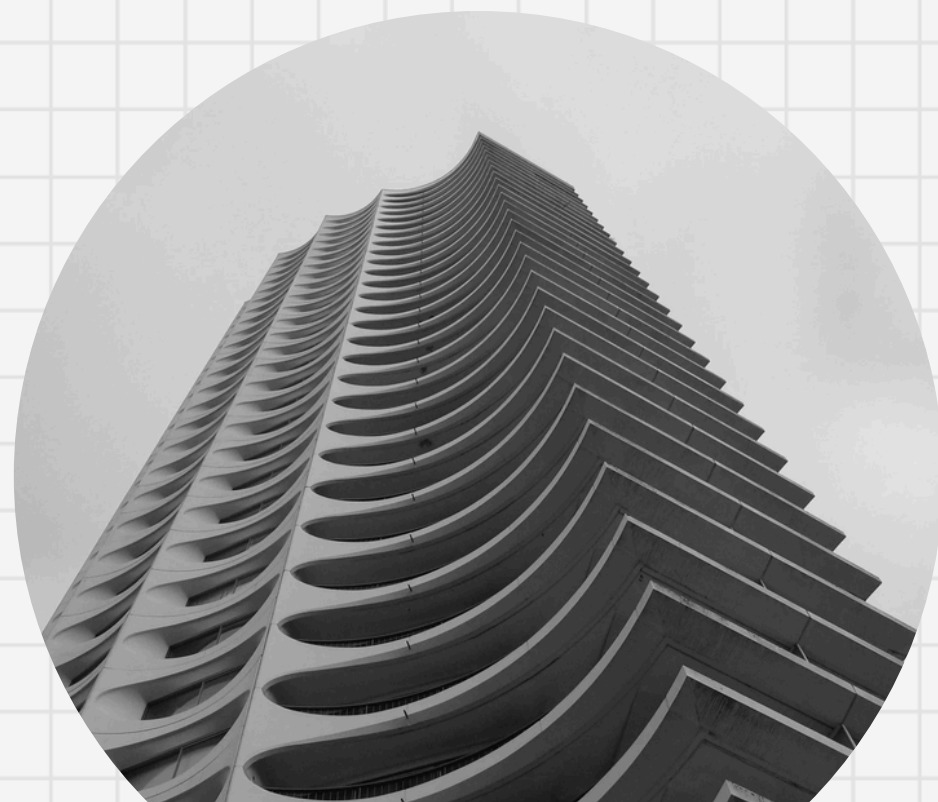




倉儲管理系統



組別：G15 何柏鋒 41143110
邱品翰 41143125



應用情境與 使用案例

應用情境

這是一套為了中小企業、電商及零件物流倉庫打造的倉儲管理系統。系統核心在於淘汰傳統 Excel 慢速對帳與儲位不明的痛點，協助管理員透過網頁系統即時掌握「特定貨品在特定儲位」的精準庫存，並完整追蹤所有入庫、出庫與庫內移庫的人員操作軌跡。

使用案例

- 入庫管理：使用者建立進貨單，並在進貨明細中記錄各貨品的進貨數量，系統隨後自動增加對應儲位管理中的貨品庫存。
- 出庫管理：使用者建立出貨單，並在出貨明細中記錄各貨品的出貨數量，系統隨後自動扣減對應儲位管理中的貨品庫存。
- 移倉作業：當貨品需要變更存放位置時，系統透過貨品移動作業記錄該貨品的起始儲位、終點儲位、移動數量與時間。
- 貨品資料管理：用於維護基本物料清單，作為進出貨、儲位存放與移動的基礎核心。
- 儲位空間管理：用於規劃倉庫物理空間，記錄每個儲位Id的區域、貨架號、儲位型態與最大容量限制，以防儲位超載。
- 庫存盤點與查詢：透過存放關係，結合貨品與儲位管理，讓使用者能即時查詢特定貨品在各儲位的貨品庫存數量。

系統需求說明

1. 系統可記錄商品入庫與出庫
2. 系統可即時計算庫存數量
3. 系統可查詢商品存放位置
4. 系統可產生庫存報表
5. 系統需支援多人同時操作
6. 系統可記錄內部移倉調撥的位置與終點

完整性限制

完整性限制類型	限制敘述
實體完整性限制	● 貨品實體：貨品ID為主鍵，不能為空值。 ● 使用者實體：使用者ID為主鍵，不能為空值。 ● 進貨實體：進貨ID為主鍵，不能為空值。 ● 出貨實體：出貨ID為主鍵，不能為空值。 ● 儲位管理實體：儲位Id 為主鍵，不能為空值。

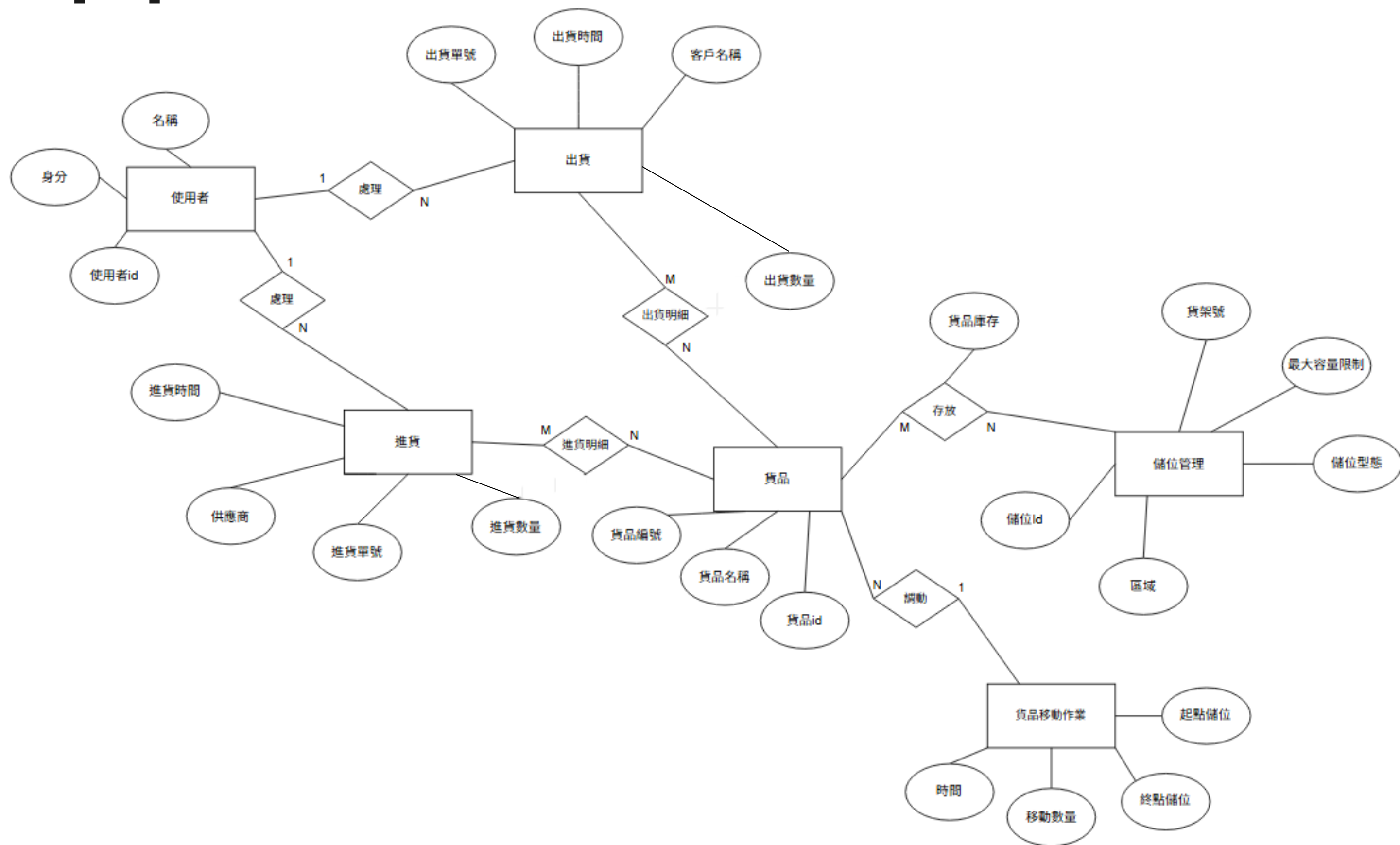
完整性限制

完整性限制類型	限制敘述
實體完整性參考 約束	• 在出貨管理關係中，每次記錄出貨時，必須選擇一個已經在貨品列表中存在的貨品。 • 在使用者關係中，每一筆進貨紀錄，必須對應到一個已經在使用者列表中存在的人員。 • 在使用者關係中，每一筆出貨紀錄，必須對應到一個已經在使用者列表中存在的人員。 • 在使用者關係中，每個貨品資訊的修改，必須對應到一個具備「管理者」身分的使用者。 • 在出貨紀錄關係中，出貨的數量必須小於或等於該貨品在貨品列表中的現有庫存量。 • 每次進行貨品調動時，所指定的 起始儲位 與 終點儲位 必須存在於儲位管理實體。

完整性限制

完整性限制類型	限制敘述
欄位完整性約束	• 所有實體ID不得為空 • 貨品庫存必須≥ 0 • 貨品位置不得為空 • 貨品名稱不得為空 • 進/出貨單號不得為空，且唯一 • 進/出貨時間不得為空 • 進/出貨數量必須 > 0 • 供應商不得為空 • 使用者ID、名稱和身分不得為空 • 貨品移動作業的起始儲位終點儲位移動數量與時間皆不得為空值。

ER圖



實體

實體名稱	屬性名稱
貨品	product_id (貨品id) product_code (貨品編號) name (貨品名稱)

實體名稱	屬性名稱
使用者	user_id (使用者id) username (名稱) role (身分)

實體名稱	屬性名稱
進貨	purchase_id (進貨單號) purchase_time (進貨時間) supplier (供應商) quantity (進貨數量)

實體名稱	屬性名稱
出貨	shipment_id (出貨單號) shipment_time (出貨時間) customer_name (客戶名稱) quantity (出貨數量)

實體

實體名稱	屬性名稱
儲位管理	location_id (儲位Id) area (區域) shelf_no (貨架號) location_type (儲位型態) max_capacity (最大容量限制)

實體名稱	屬性名稱
儲位管理	start_location (起始儲位) end_location (終點儲位) quantity (移動數量) move_time (時間)

關係

關係名稱	敘述
進貨	• 一個使用者可以處理多筆進貨 • 一筆進貨只能由一個使用者處理
出貨	• 一個使用者可以處理多筆出貨 • 一筆出貨只能由一個使用者處理
存放	• 一個貨品可以分開存放於多個儲位 • 一個儲位可以同時存放多種貨品
調動	• 一個儲位可以參與多次貨品移動 • 一次貨品移動作業只能對應一個特定的儲位

資料表

```
CREATE TABLE User (  
  user_id VARCHAR(10) PRIMARY KEY,  
  username VARCHAR(10) NOT NULL,  
  role VARCHAR(10) NOT NULL  
);使用者
```

```
CREATE TABLE Product (  
  product_id VARCHAR(10) PRIMARY KEY,  
  product_code VARCHAR(10) NOT NULL UNIQUE,  
  name VARCHAR(10) NOT NULL  
);貨品
```

```
CREATE TABLE Location (  
  location_id VARCHAR(10) PRIMARY KEY,  
  area VARCHAR(10) NOT NULL,  
  shelf_no VARCHAR(10) NOT NULL,  
  location_type VARCHAR(10) NOT NULL,  
  max_capacity INT NOT NULL CHECK(max_capacity>0)  
);儲位管理
```

```
CREATE TABLE Purchase (  
  purchase_id VARCHAR(10) PRIMARY KEY,  
  purchase_time DATETIME NOT NULL,  
  supplier VARCHAR(10) NOT NULL,  
  user_id VARCHAR(10) NOT NULL,  
  FOREIGN KEY (user_id)  
  REFERENCES User(user_id)  
);進貨
```



```
CREATE TABLE Shipment (  
shipment_id VARCHAR(10) PRIMARY KEY,  
shipment_time DATETIME,  
customer_name VARCHAR(10) NOT NULL,  
user_id VARCHAR(10) NOT NULL,  
);出貨
```

```
CREATE TABLE Purchase_Detail (  
purchase_id VARCHAR(10) NOT NULL,  
product_id VARCHAR(10) NOT NULL,  
quantity INT ,  
user_id VARCHAR(10) NOT NULL,  
PRIMARY KEY (purchase_id, product_id)  
);進貨明細
```

```
CREATE TABLE Product_Movement (  
movement_id INT AUTO_INCREMENT PRIMARY KEY,  
product_id VARCHAR(10) NOT NULL,  
start_location VARCHAR(10) NOT NULL,  
end_location VARCHAR(10) NOT NULL,  
quantity INT NOT NULL, move_time DATETIME DEFAULT CURRENT_TIMESTAMP,  
user_id VARCHAR(10) NOT NULL,  
FOREIGN KEY (product_id) REFERENCES Product(product_id),  
FOREIGN KEY (start_location) REFERENCES Location(location_id),  
FOREIGN KEY (end_location) REFERENCES Location(location_id),  
FOREIGN KEY (user_id) REFERENCES User(user_id)  
);
```

```
CREATE TABLE Shipment_Detail (  
    shipment_id VARCHAR(15) NOT NULL,  
    product_id VARCHAR(10) NOT NULL,  
    quantity INT NOT NULL,  
    user_id VARCHAR(10) NOT NULL,  
    PRIMARY KEY (shipment_id, product_id),  
    FOREIGN KEY (shipment_id) REFERENCES Shipment(shipment_id),  
    FOREIGN KEY (product_id) REFERENCES Product(product_id),  
    FOREIGN KEY (user_id) REFERENCES User(user_id)  
);
```



```
CREATE TABLE Stock (  
    product_id VARCHAR(10) NOT NULL,  
    location_id VARCHAR(10) NOT NULL,  
    stock_quantity INT NOT NULL,  
    PRIMARY KEY (product_id, location_id),  
    FOREIGN KEY (product_id) REFERENCES Product(product_id),  
    FOREIGN KEY (location_id) REFERENCES Location(location_id)  
);
```

VIEW

產品儲存位置查詢

```
1 CREATE VIEW View_Inventory_Status AS
2 SELECT
3     p.name AS 產品名稱,
4     l.area AS 區域,
5     l.shelf_no AS 儲位,
6     s.stock_quantity AS 數量
7 FROM Stock s
8 JOIN Product p ON s.product_id = p.product_id
9 JOIN Location l ON s.location_id = l.location_id;
```

進貨作業

```
CREATE VIEW View_Purchase_Performance AS
SELECT
    p.purchase_time AS 進貨時間,
    p.supplier AS 供應商,
    u.username AS 經辦人員,
    pr.name AS 產品名稱,
    pd.quantity AS 進貨數量
FROM Purchase p
JOIN Purchase_Detail pd ON p.purchase_id = pd.purchase_id
JOIN Product pr ON pd.product_id = pr.product_id
JOIN User u ON p.user_id = u.user_id;
```

貨品移動作業

```
CREATE OR REPLACE VIEW view_product_flow AS
SELECT
    p.name AS product_name,
    l1.location_type AS start_area,
    l2.location_type AS end_area,
    pm.quantity AS move_qty,
    pm.move_time AS move_time,
    u.username AS staff_name
FROM product_movement pm
JOIN product p ON pm.product_id = p.product_id
JOIN location l1 ON pm.start_location = l1.location_id
JOIN location l2 ON pm.end_location = l2.location_id
JOIN user u ON pm.user_id = u.user_id;
```

參考資料

GeeksforGeeks. (2024, May 14). MySQL CREATE INDEX Statement. GeeksforGeeks.
<https://www.geeksforgeeks.org/mysql-create-index-statement/>

iT 邦幫忙. (2020). 實體關聯性模型圖 ER/EER Diagram. iT 邦幫忙::一起幫忙解決難題，拯救 IT 人的一天.
<https://ithelp.ithome.com.tw/articles/10237077>

W3Schools. (n.d.). SQL INSERT INTO Statement. W3Schools Online Web Tutorials.
https://www.w3schools.com/sql/sql_insert.asp

心得報告

我們把課堂上那些的資料庫理論和操作，使用 SQL 寫了出來。從資料表的新增、修改，還學會用 View 把好幾張表 JOIN 在一起，直接解決了多品項進出貨的對帳麻煩。

而且因為我們把員工編號加進明細表，現在系統一查就能抓出誰在什麼時間動了庫存，這才讓我發現寫好 SQL、設計好資料庫結構，對公司的資產管理和資料正確性有多重要。

41143110 何柏鋒